Title

Social Media-Based Travel Trend Prediction with Deepfake Content Filtering

Motivation

These days, people don't pick a travel destination from guidebooks — a lot of it comes from what they see on Instagram, YouTube, or Twitter. One viral reel can turn a quiet, unknown place into a major tourist spot within weeks, so tourism boards and travel companies keep a close eye on social media to spot which destinations are gaining attraction.

The problem is, not everything that goes viral is real. With AI image and video tools now so easy to use, fake or manipulated travel content — a stunning "photo" of a place that was never actually taken there — is increasingly common online. If this is treated the same as genuine content, it can throw off any system trying to measure real interest in a destination.

That's the gap this project addresses — a system that doesn't just track social media buzz, but also checks whether that buzz is coming from real content. Instead of treating trend prediction and deepfake detection as separate problems, this project brings them together so the forecasting is actually backed by trustworthy data.

Objectives

1. Collect google trends activity data — mentions, hashtags, and search trends — for different travel destinations.
2. Build a model that can forecast whether a destination's popularity is likely to go up or down in the near future.
3. Build a separate module that can look at travel-related images and flag whether they're real or AI-manipulated.
4. Connect the two modules so that only content confirmed to be genuine is used while calculating buzz and generating trend predictions.
5. Compare the trend predictions with and without the deepfake filter in place, to see whether removing fake content actually makes a noticeable difference to the results.

Modules

1. Data Collection Module
This module handles gathering the two kinds of data the project needs — social media activity and mentions around different travel destinations, and a set of real and AI-manipulated travel images/videos that the detection model will later learn from.

2. Deepfake Detection Module
This is basically the "fact-checking" part of the system. Whenever an image or video tied to a trending destination comes in, this module looks at it and flags whether it seems genuine or AI-generated/tampered with, before it's allowed to move forward.

3. Scoring Module
After the fake content has been filtered out, this part looks at what's left — how often a destination is being mentioned and whether people are talking about it positively or negatively — and combines both into a single score.

4. Trend Forecasting Module
Using the scores as history, this is the part that actually makes the prediction — whether interest in a place is likely to grow, shrink, or stay flat in the coming weeks. This is the core forecasting engine of the project.

5. Integration Layer
This is what ties everything together. Social content comes in, passes through the fake-detection check first, and only the verified, genuine content moves on to the trend prediction side — so the two modules work as one connected pipeline rather than separately.

6. Dashboard/Visualization Module
This is where everything comes together visually — trending destinations, how their buzz has changed over time, and how much fake content got caught along the way — turning raw output into something a guide or evaluator can actually look at and understand.

Methodology

Here's how the project will actually be carried out, step by step, module by module:

1. Data Collection
The first step is pulling together data from two different places. On one side, destination-related activity is collected from social platforms — things like how often a place is mentioned, which hashtags are being used, and how that changes week to week. On the other side, a separate dataset of real and fake travel images/videos is gathered so the detection model has something to learn from. These two collection efforts run more or less independently since they feed two different parts of the system.

2. Preprocessing and Cleaning
Raw data from social media is messy — duplicate posts, spam, irrelevant mentions, missing values, and so on. Before anything useful can be done with it, this data needs to be cleaned up: removing duplicates, filtering out irrelevant content, and structuring it into a consistent format (like mentions-per-day per destination). Similarly, the image/video dataset needs to be resized, normalized, and labeled properly before it can be used to train the detection model.

3. Deepfake Detection
Once the visual data is ready, a detection model is trained to tell real content apart from manipulated content. This involves feeding the model examples of both real and fake images/videos so it learns the visual patterns that usually give away manipulation — things like unnatural blending, inconsistent lighting, or facial artifacts. Once trained, this model is used to check new incoming destination-related media and decide whether it's genuine or not.

4. Trend Forecasting
With scores calculated over time, this becomes a time-series problem — essentially looking at the pattern of a destination's popularity so far and predicting where it's headed next. This step involves training a forecasting model on historical buzz data so it can pick up on rising or falling patterns and project them a few weeks into the future.

5. Integration
Instead of treating detection and forecasting as two separate scripts, this step connects them into one flow — new content comes in, gets checked by the detection model first, and only what passes that check gets used to update the buzz score and feed the forecasting model. This is what actually makes it one integrated system rather than two disconnected projects.

6. Evaluation
Finally, the results are evaluated in two ways — checking how accurately the detection model tells real from fake, and checking how close the forecasting model's predictions are to actual trend movement. As an extra step, predictions made with the deepfake filter in place can be compared to predictions made without it, to actually show whether filtering out fake content makes a real difference.

Technologies

Programming language

- Python — used across the entire project, from data collection scripts to model training to the dashboard, since it has the strongest ecosystem for both NLP/time-series work and deep learning.

Data Collection

Python — Chosen because nearly every platform used here (Twitter, Google Trends) has an actively maintained Python client. This avoids handling raw HTTP requests and authentication manually. It also keeps the entire pipeline, from collection to modeling, in a single language.

pytrends — Used to pull Google Trends data programmatically instead of manually downloading it. Google has no official public API for Trends, so pytrends is the standard community-built wrapper for this. It makes it easy to automate collection across multiple destinations and dates.

Data Processing

pandas — Used to clean and structure the raw mentions data into a usable format, like counting mentions per destination per day. It's the standard library for tabular data handling in Python, with fast, well-tested functions for exactly this kind of cleanup. Almost every other library in this stack (scikit-learn, Prophet) expects data in pandas format, so it fits naturally into the pipeline.

NumPy — Used alongside pandas for any numerical operations, like normalizing counts or handling missing values mathematically. It's the underlying numerical engine pandas itself is built on, so using it directly is efficient for anything pandas doesn't handle out of the box.

Trend Forecasting

scikit-learn — Used to build baseline regression models for forecasting, as a simpler starting point before trying more specialized time-series tools. It's the most widely used ML library in Python, with consistent, easy-to-use interfaces for training and testing models. It also fits well here since it integrates directly with pandas dataframes already used earlier in the pipeline.

Prophet — Used as the main forecasting model, since it's built specifically for time-series data with trend and seasonality patterns, which is exactly what destination buzz data looks like over weeks and months. It's designed by Meta to be easy to use even without deep time-series expertise, requiring minimal tuning to get reasonable results.

statsmodels (ARIMA) — Used as a classical statistical alternative to compare against Prophet's results. It's a well-established, traditional approach to time-series forecasting that's often expected in academic settings as a baseline comparison. Including it shows the model choice was evaluated against a standard method, rather than picking just one approach without justification.

Deepfake Detection Model

TensorFlow / Keras — This is like a toolkit used to actually build and teach the model that learns to spot fake images. Keras is picked because it's simpler to write and understand — you can build a working model in fewer lines of code. It's like choosing the easier, more beginner-friendly tool to get the job done without needing to write everything from scratch.

XceptionNet — Instead of teaching the model everything from zero, this is like starting with a model that has already learned to recognize patterns in thousands of images. It's already good at spotting the small, tricky signs that an image has been faked or altered, so you don't have to train it from the ground up.

Integration and Dashboard

Python (functions/scripts) — Used to connect the deepfake detection module and the buzz/forecasting module into one pipeline, where the detector's real/fake output directly decides what data moves forward. No separate integration framework is needed here since both modules are already built in Python — a set of connecting functions is enough to link them without adding extra tools or complexity.

Streamlit — Used to build the dashboard that displays trending destinations, buzz scores over time, and how much fake content was filtered out. It's chosen over a full web framework like Flask or Django because it's built specifically for quickly turning Python scripts/data into an interactive dashboard, without needing separate frontend code. This fits here well since the goal is a functional, readable output rather than a production-grade web app.

Power BI (optional) — Can optionally be used as a secondary, presentation-layer dashboard alongside Streamlit. The Python pipeline would export its final processed output (buzz scores, trend predictions, flagged-content counts) as a CSV file or push it into a simple database like SQLite, which Power BI can then connect to directly to build polished visuals, cards, and charts on top of it. This wouldn't replace Streamlit as the working dashboard, since Power BI isn't connected live to the Python code and would need the data re-exported or refreshed each time the model runs again, but it works well as an extra, presentation-ready view to show alongside the project.

Development environment

- Google Colab / Jupyter Notebook — for training models (Colab gives free GPU access, useful for the deepfake detection part)
- GitHub — for version control and maintaining your project repository

Dataset

Since this project actually needs two different kinds of data for its two models, here's where each one comes from and how it can realistically be gathered.

For the Trend Forecasting Model

This model needs to know how much a destination is being talked about over time, so the data here is basically activity/mention data pulled from platforms where travel conversations actually happen.

- **Google Trends** — This is probably the easiest and most reliable place to start. It shows search interest for any destination name over weeks or months, and it's completely free with no approval needed. There's a Python library called pytrends that pulls this data automatically, so it can be scripted rather than downloaded manually.

For the Deepfake Detection Model

- **CIFAKE** — A ready-made dataset of real vs. AI-generated images (not face-specific), small and manageable in size, making it a good base dataset for a semester project.
- **Custom destination-specific set (recommended as the core of this module)** — Since no existing public dataset is built specifically around "real vs. fake travel destination photos," the most accurate approach is to build a small dataset directly:
    - **Real images**: pulled from official tourism board websites, or free stock photo sources like Unsplash and Pexels.
    - **Fake images**: generated using free AI image tools such as Stable Diffusion (via Hugging Face or Google Colab) by prompting for similar destinations/landmarks or downloaded from fake ai generating websites.
    - These are labeled real/fake and used as the actual training and testing set for the detector.

Trend forecasting data collection: Destination-related activity is pulled from platforms like Google Trends, capturing mentions, hashtags, and search interest for each place over time. This raw data is then cleaned to remove duplicates and irrelevant posts, and organized into a day-wise mentions table per destination. The end result is a time-series dataset that captures how a destination's online popularity changes week by week, which is then fed into the forecasting model.

Deepfake detection data collection: Two sets of image data are brought together for this module. General datasets like CIFAKE, which contain large numbers of labeled real and AI-generated images, are used to teach the model basic patterns that separate authentic images

from generated ones. Alongside this, a smaller custom dataset is built specifically for the travel context — real destination photos are sourced from official tourism websites, while fake destination images are generated using tools like Stable Diffusion or Hugging Face and downloaded using other websites. The model is first trained on the general dataset and then fine-tuned on this destination-specific set, resulting in a detector that is tailored to catch fake travel content rather than just generic AI images.